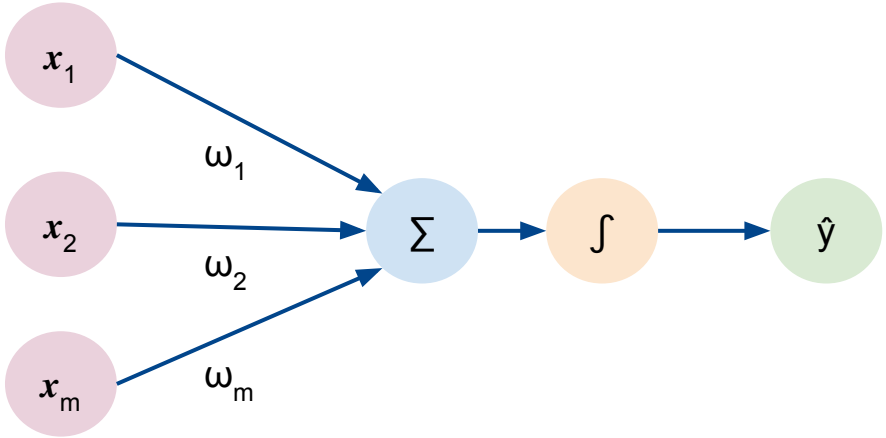
EEE 6778. Applied Machine Learning II

Module 1. Introduction to Deep Learning and Optimization Techniques

The Perceptron
Neural Networks built from Perceptrons

The Perceptron

The Perceptron: Forward Propagation



Inputs Weights Sum Non-linearity Output

Output

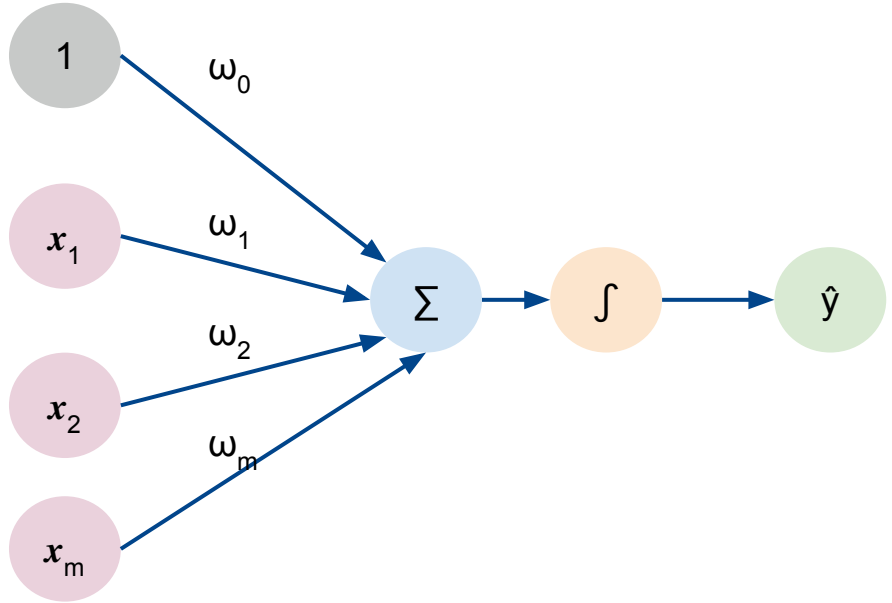
Linear combination of outputs

$$\hat{y} = g\left(w_0 + \sum_{i=1}^{m} x_i w_i\right)$$

Non-Linear Activation function

(Amini, 2023)

The Perceptron: Forward Propagation



Output

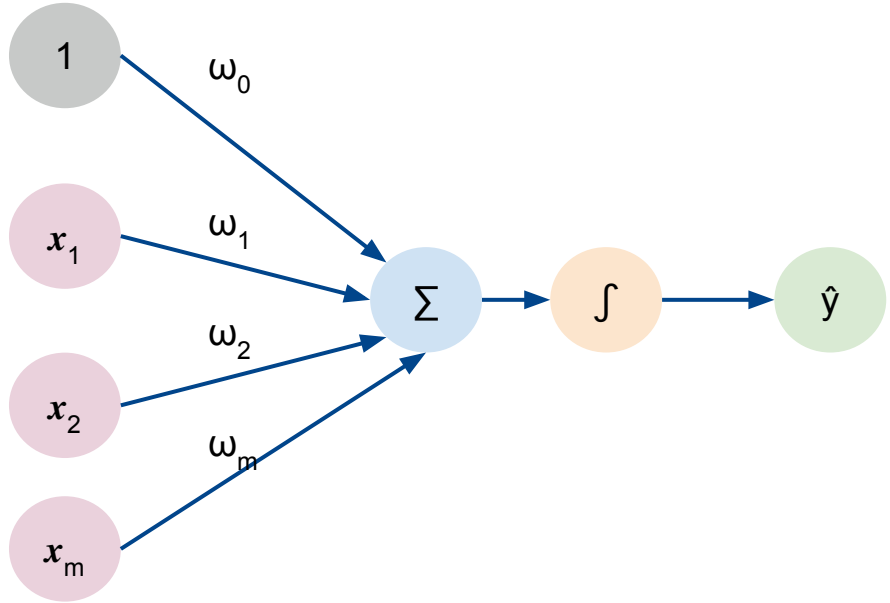
Linear combination of outputs

$$\hat{y} = g\left(\sum_{i=1}^{m} x_i w_i + b\right)$$

Non-Linear Activation function

(Amini, 2023)

The Perceptron: Forward Propagation



$$\hat{y} = g\left(\sum_{i=1}^{m} x_i w_i + b\right)$$

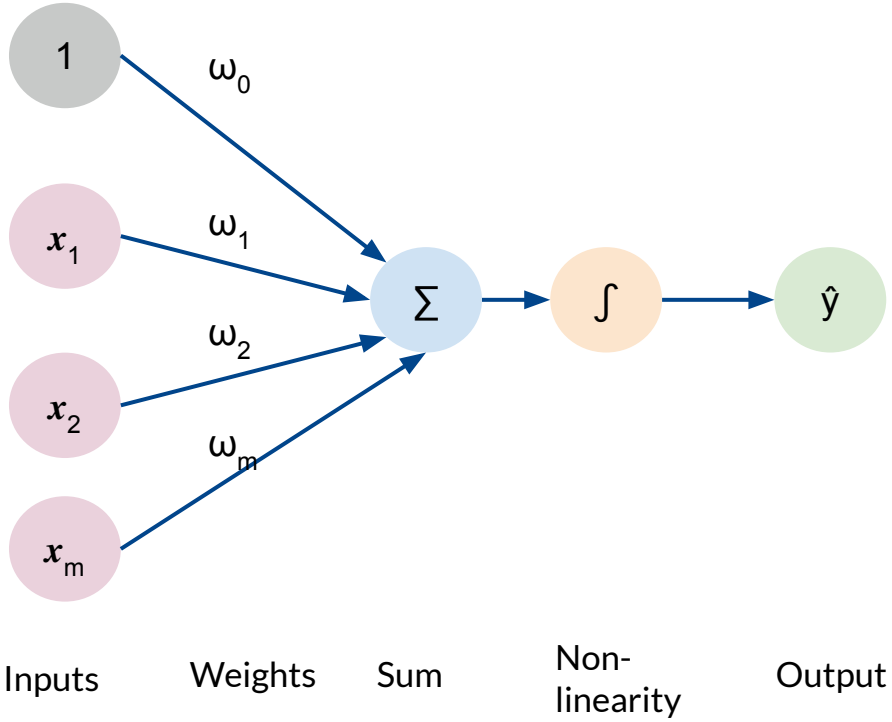
$$\hat{y} = g\left(w_0 + \mathbf{X}^T\mathbf{W}\right)$$

Where

$$\mathbf{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \quad \text{and} \quad \mathbf{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

(Amini, 2023)

The Perceptron: Forward Propagation

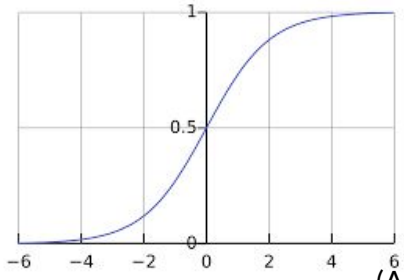


Activation Functions

$$\hat{y} = g\left(w_0 + \mathbf{X}^T\mathbf{W}\right)$$

Example: sigmoid function

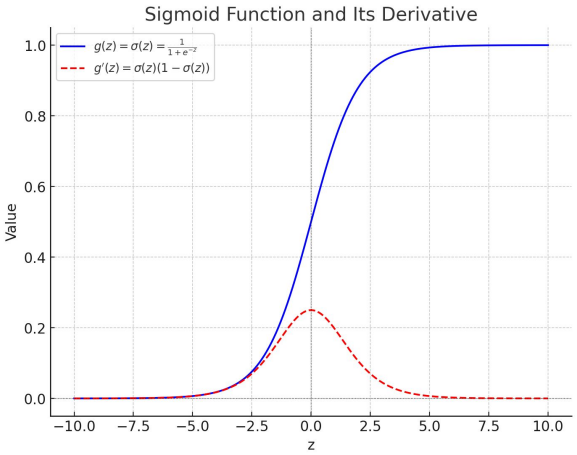
$$g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$



(Amini, 2023)

Common Activation Functions

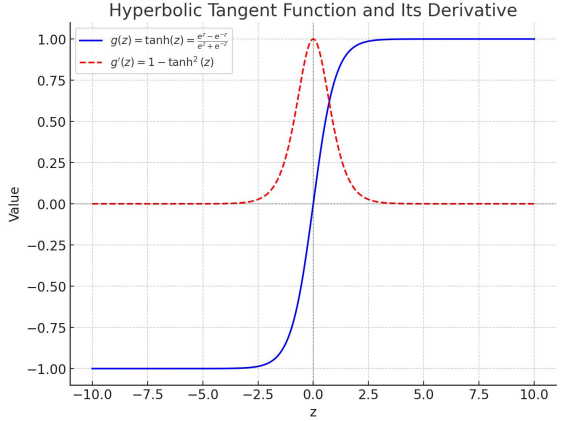
Sigmoid Function



$$g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$

$$g'(z) = \sigma'(z) = \sigma(z)\left(1-\sigma(z)\right)$$

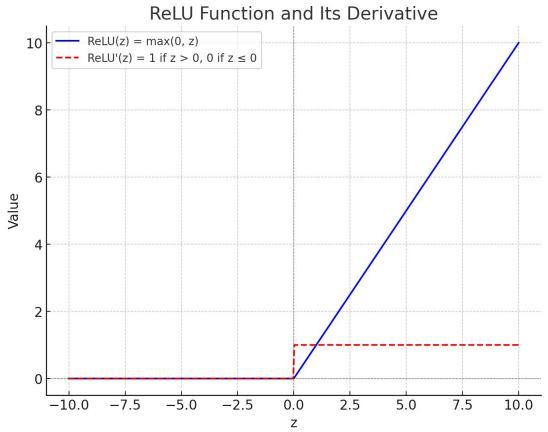
Hyperbolic



$$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - \tanh^2(z) = 1 - g^2(z)$$

Rectified Linear Unit (RELU)



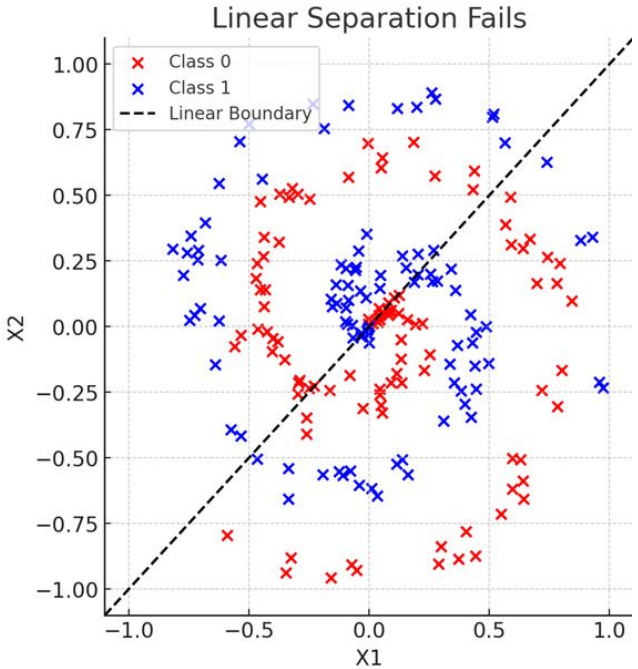
$$g(z) = \mathrm{ReLU}(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

(Amini, 2023)

Importance of Activation Functions

- All activation functions are nonlinear
- The purpose of activation functions is to introduce non-linearities into the network

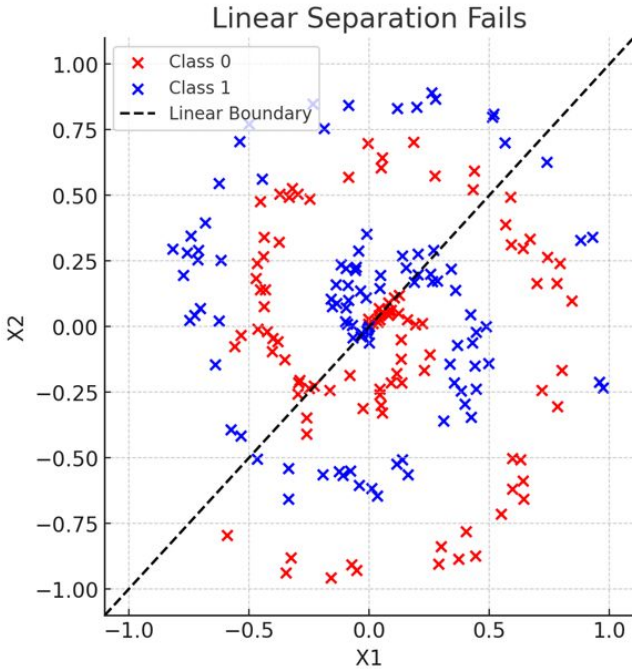


Imagine trying to separate red and blue dots.

(Amini, 2023)

Importance of Activation Functions

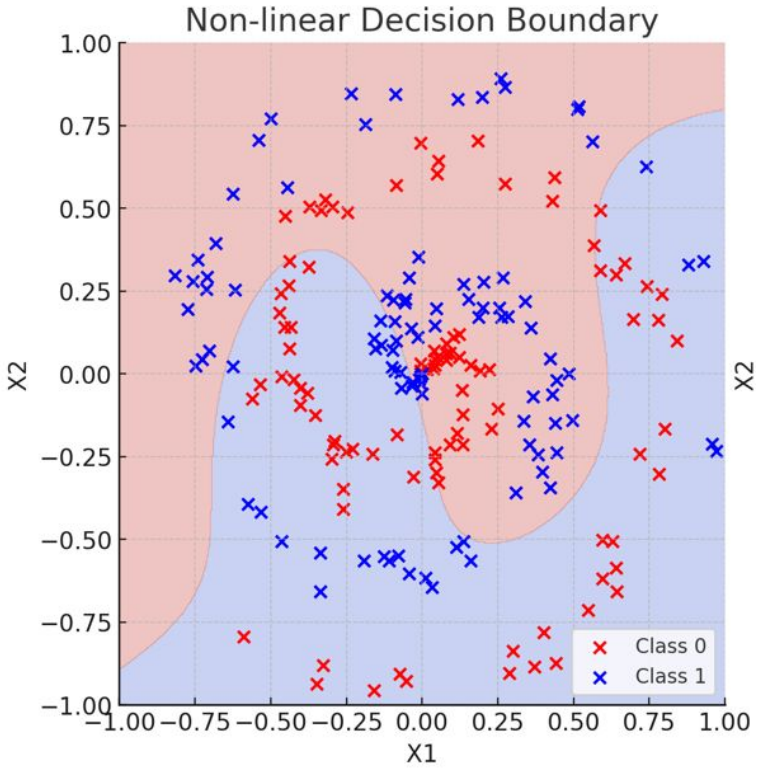
- Linear activation functions produce linear decisions no matter the network size
  - If they are mixed in a spiral pattern, a straight line won't work. The network needs to **draw curves** to separate them properly



(Amini, 2023)

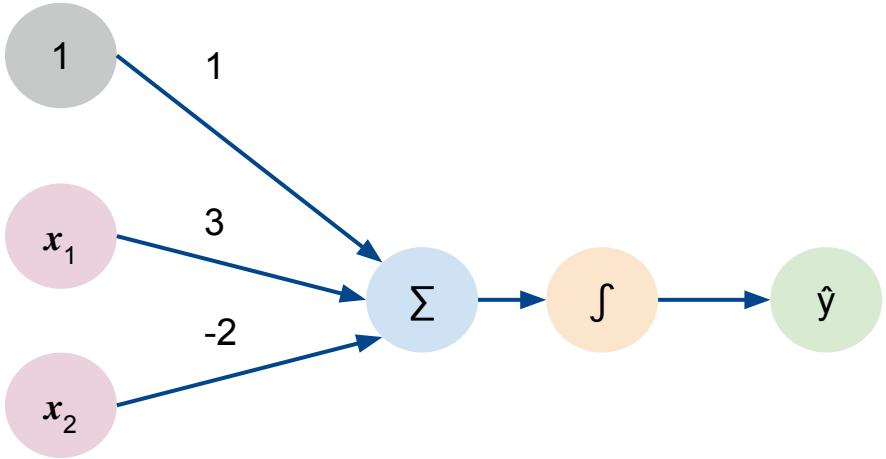
Importance of Activation Functions

- Non linearity allows to approximate arbitrarily complex functions



(Amini, 2023)

The Perceptron: Example



We have:

$$w_0 = 1 \quad \text{and} \quad \mathbf{W} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$$

$$\hat{y} = g\left(w_0 + \mathbf{X}^T\mathbf{W}\right)$$

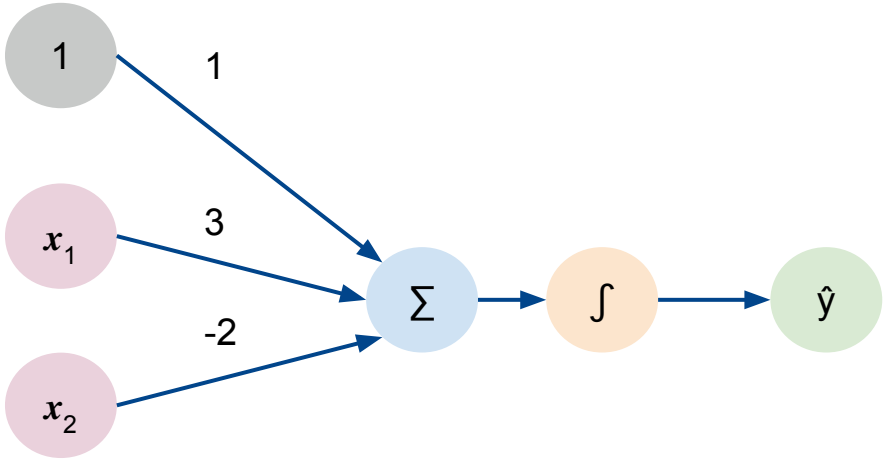
$$= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right)$$

$$\hat{y} = g\left(1 + 3x_1 - 2x_2\right)$$

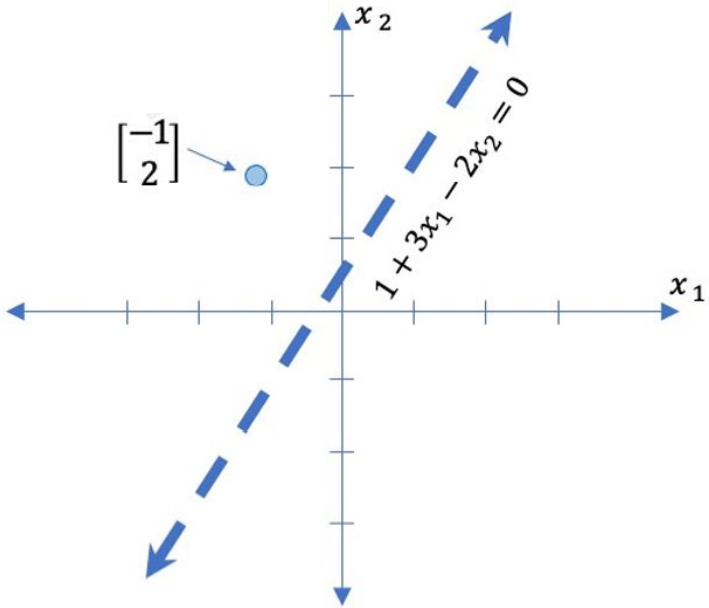
This is just a line in 2D

(Amini, 2023)

The Perceptron: Example



$$\hat{y} = g\left(1 + 3x_1 - 2x_2\right)$$



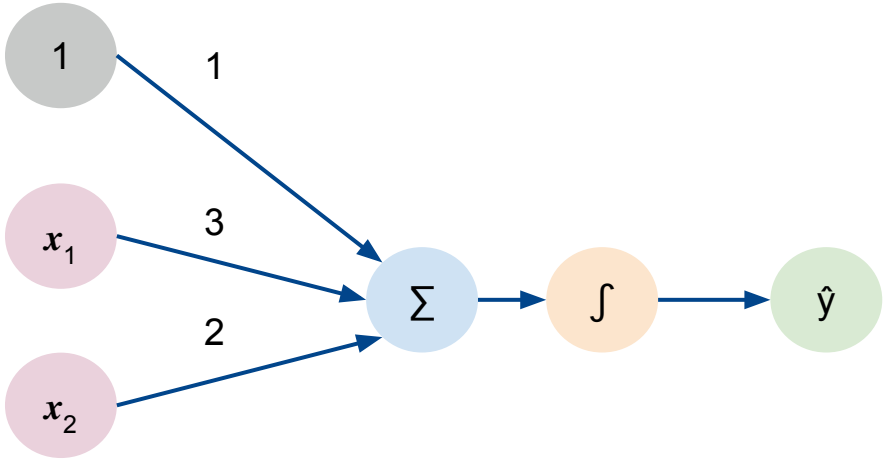
Assume we have the input $\mathbf{x} = \begin{bmatrix}-1\\2\end{bmatrix}$

$$\hat{y} = g\left(1 + (3 \times -1) - (2 \times 2)\right)$$

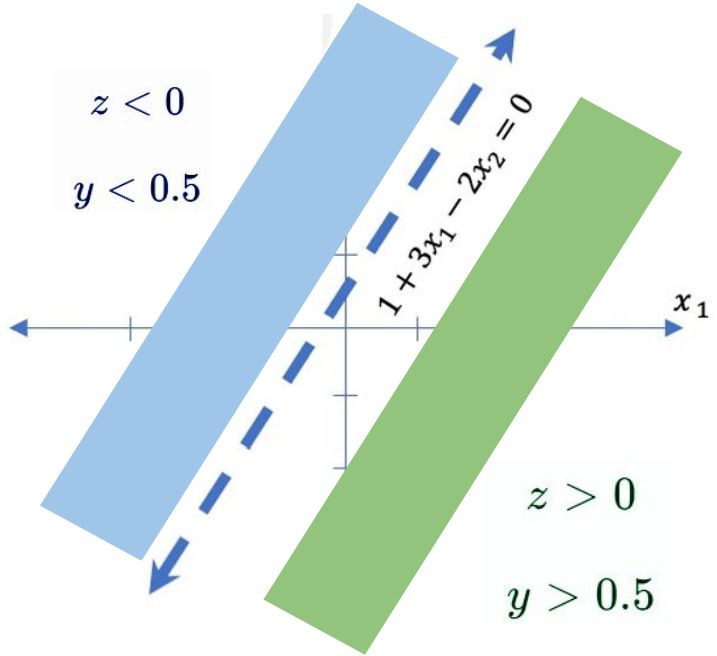
$$\hat{y} = g(-6) \approx 0.002$$

(Amini, 2023)

The Perceptron: Example



$$\hat{y} = g\,(1 + 3x_1 - 2x_2)$$

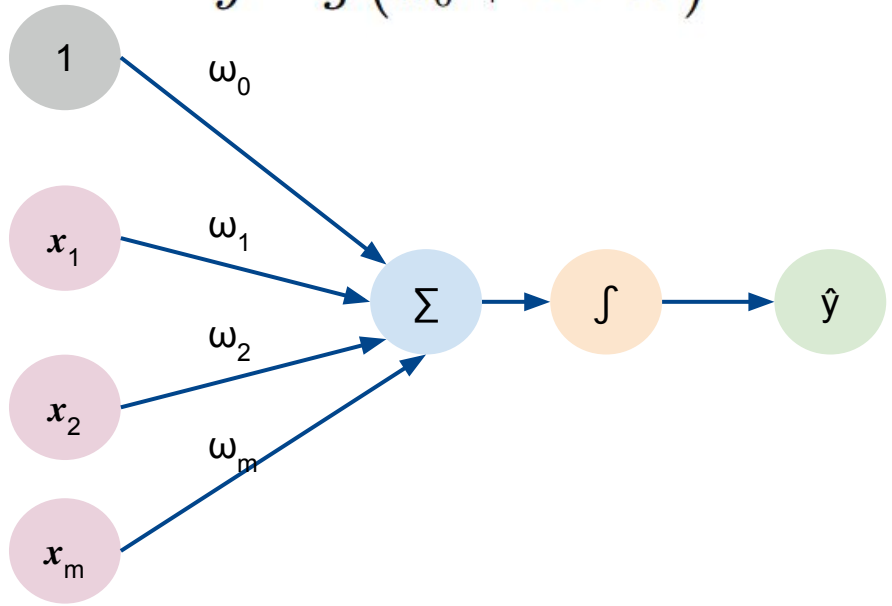


(Amini, 2023)

Neural Networks built from Perceptrons

The Perceptron Simplified

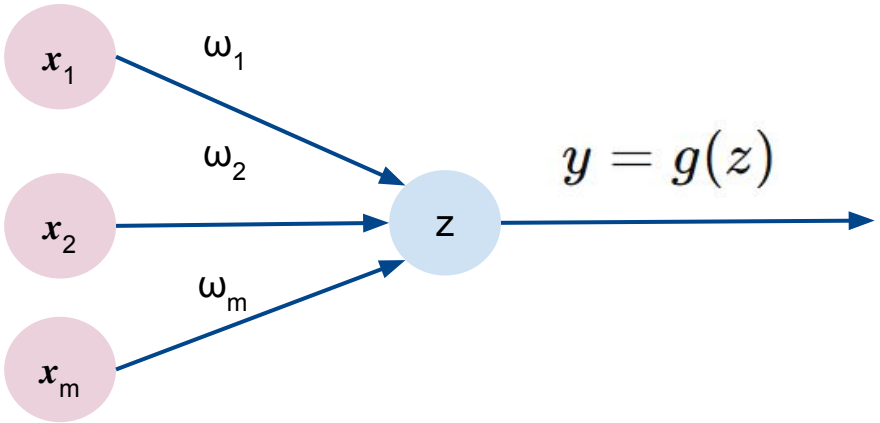
$$\hat{y} = g\left(w_0 + \mathbf{X}^T\mathbf{W}\right)$$



(Amini, 2023)

The Perceptron Simplified

$$\hat{y} = g\left(w_0 + \mathbf{X}^T\mathbf{W}\right)$$

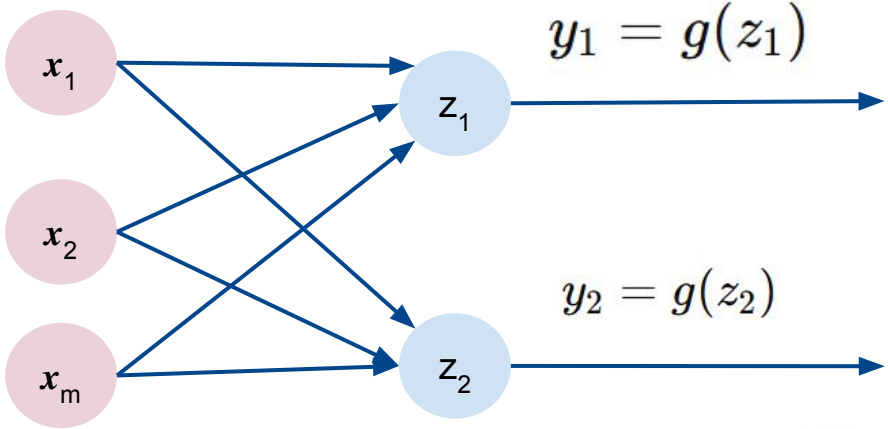


$$z = w_0 + \sum_{j=1}^{m} x_j w_j$$

(Amini, 2023)

Multi Output Perceptron

- Because all inputs are densely connected to all outputs, these layers are called **Dense** layers



$$z_1 = w_{0,1} + \sum_{j=1}^{m} x_j w_{j,1}$$

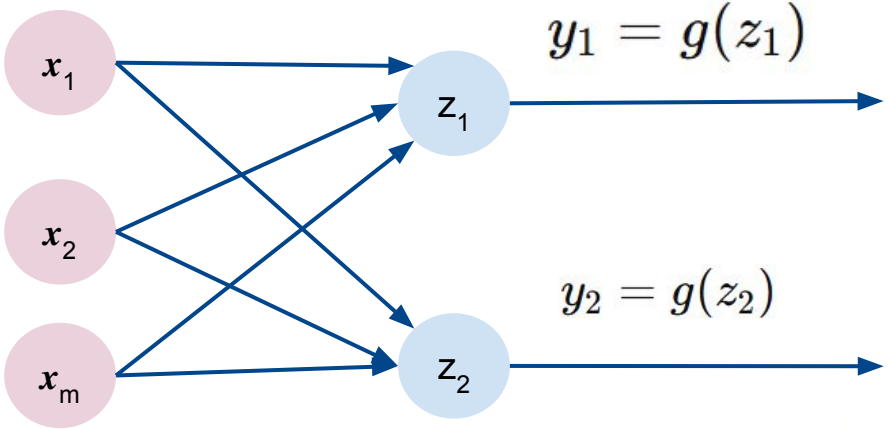
$$z_2 = w_{0,2} + \sum_{j=1}^{m} x_j w_{j,2}$$

$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j w_{j,i}$$

(Amini, 2023)

Multi Output Perceptron

- Because all inputs are densely connected to all outputs, these layers are called **Dense** layers



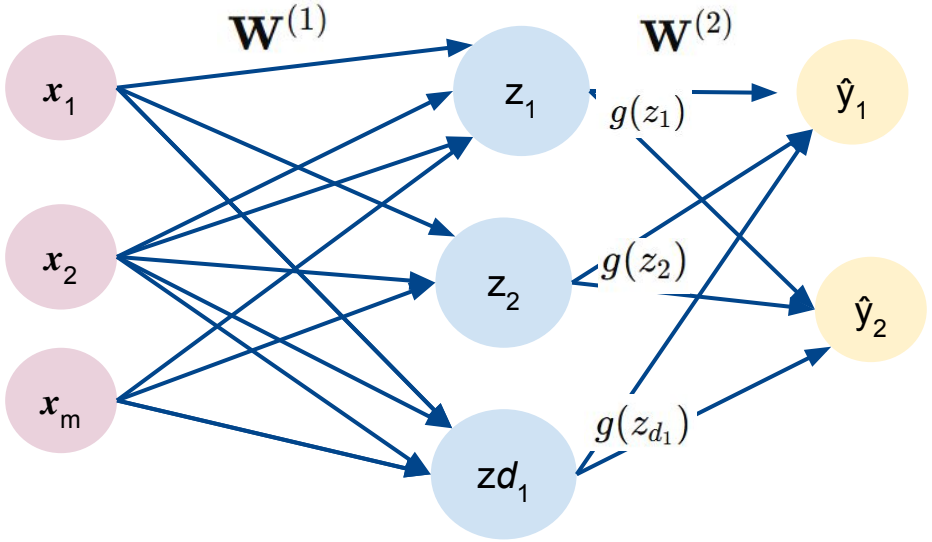

```
import torch
import torch.nn as nn

layer = nn.Linear(in_features, 2)
```

$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j w_{j,i}$$

(Amini, 2023)

Single Layer Neural Network



Inputs

Hidden Layer

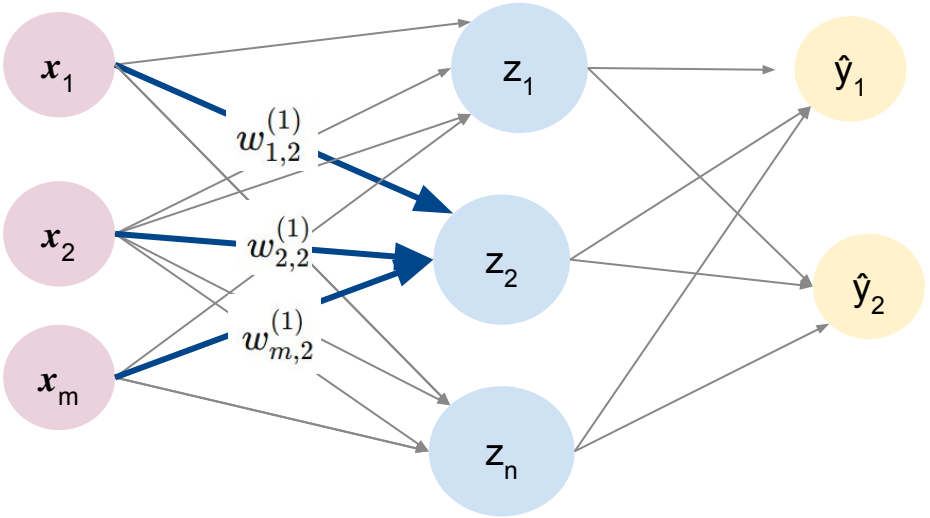
$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^{m} x_j \, w_{j,i}^{(1)}$$

Final Output

$$\hat{y}_i = g\left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) \, w_{j,i}^{(2)}\right)$$

(Amini, 2023)

Single Layer Neural Network

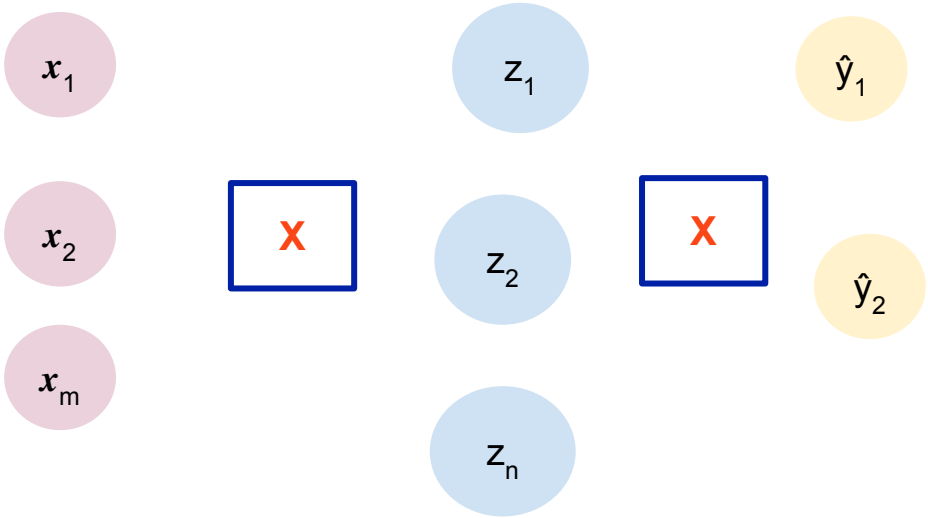


$$z_2 = w^{(1)}_{0,2} + \sum_{j=1}^{m} x_j \, w^{(1)}_{j,2}$$

$$= w^{(1)}_{0,2} + x_1 \, w^{(1)}_{1,2} + x_2 \, w^{(1)}_{2,2} + \cdots + x_m \, w^{(1)}_{m,2}$$

(Amini, 2023)

Multi Output Perceptron



Inputs

Hidden Layer

Final Output

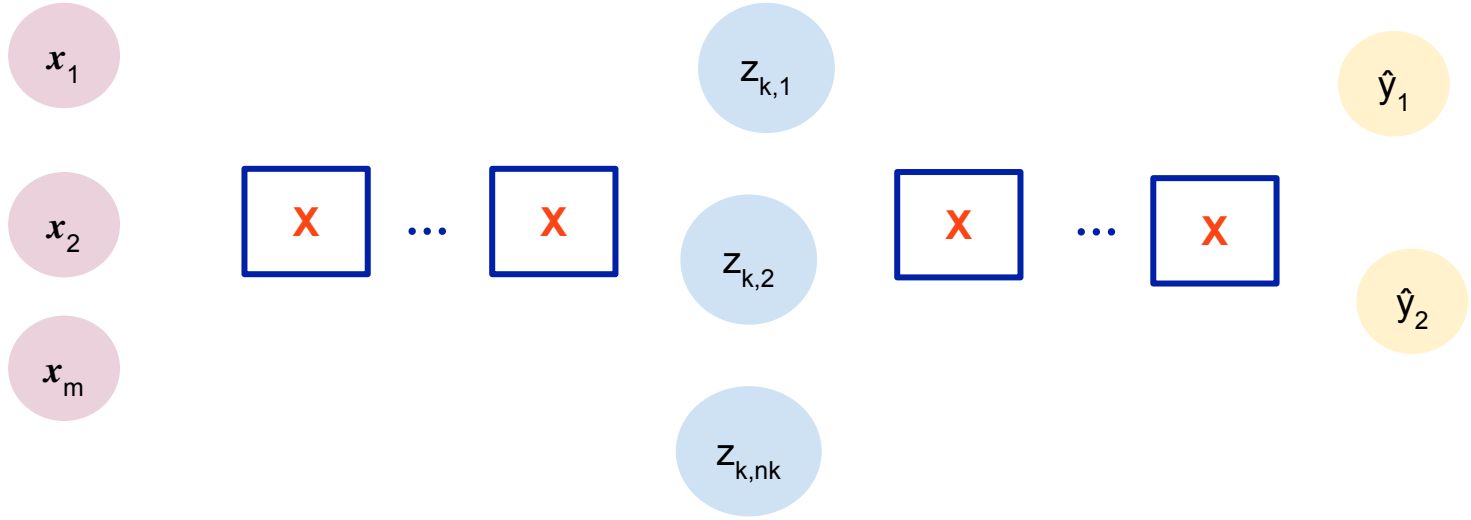
```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(input_size, n),
    nn.Linear(n, 2)
)
```

First dense layer with n units and second dense layer with 2 units

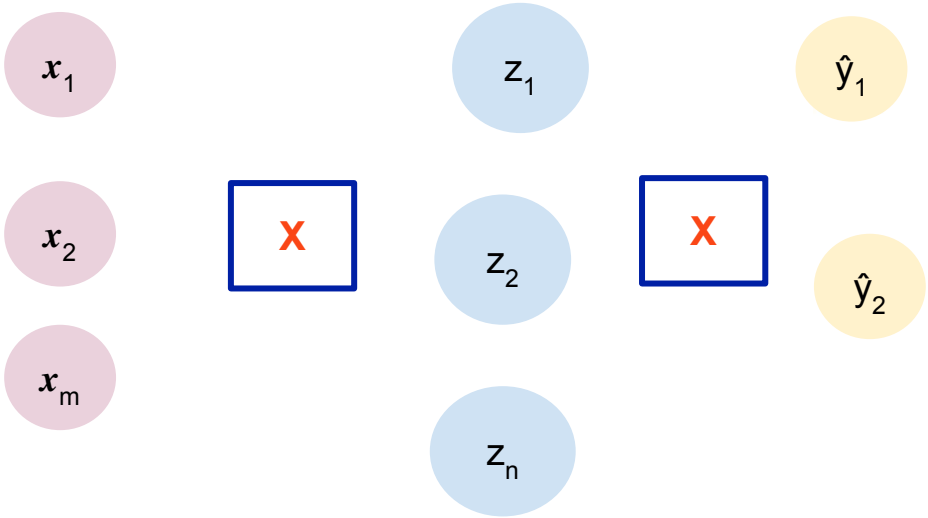
(Amini, 2023)

Deep Neural Network



(Amini, 2023)

Multi Output Perceptron



Inputs

Hidden Layer

Final Output

```
from torch import nn

model = nn.Sequential(
    nn.Linear(m, n),
    nn.ReLU(),
    nn.Linear(n, 2)
)
```

First dense layer with n units and second dense layer with 2 units

(Amini, 2023)

Example

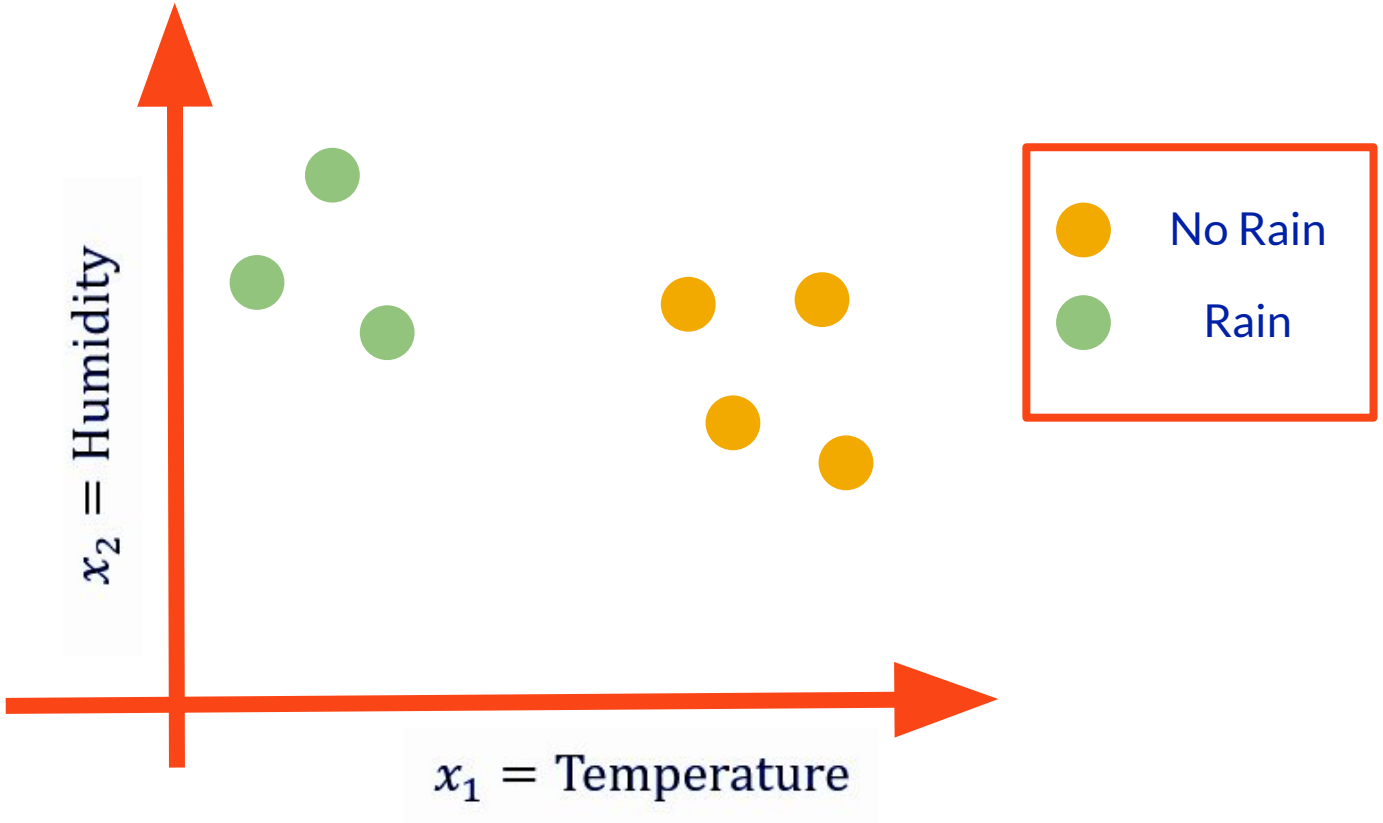
Will it rain today?

Let's start with a simple two feature model

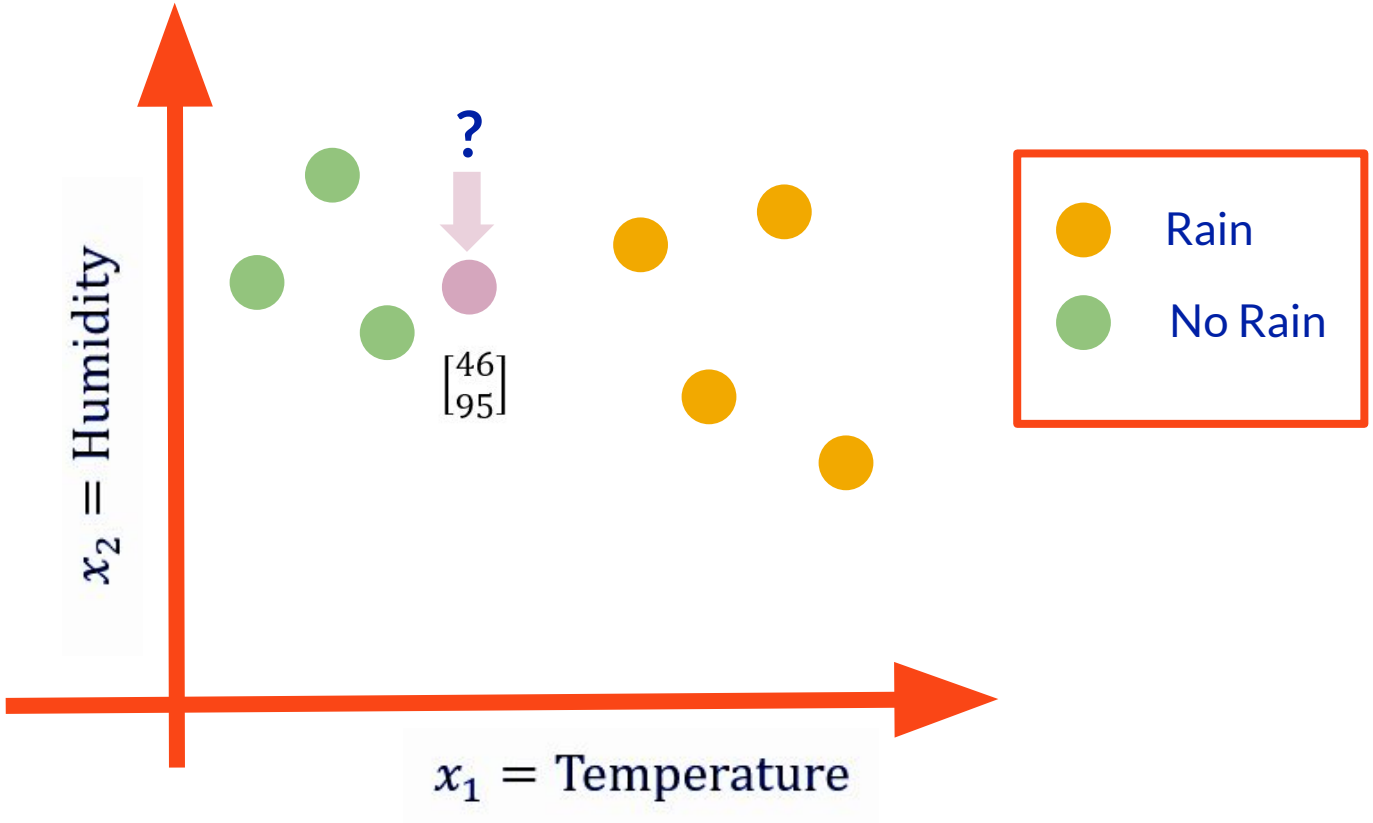
$$x_1 = \text{Temperature}$$

$$x_2 = \text{Humidity}$$

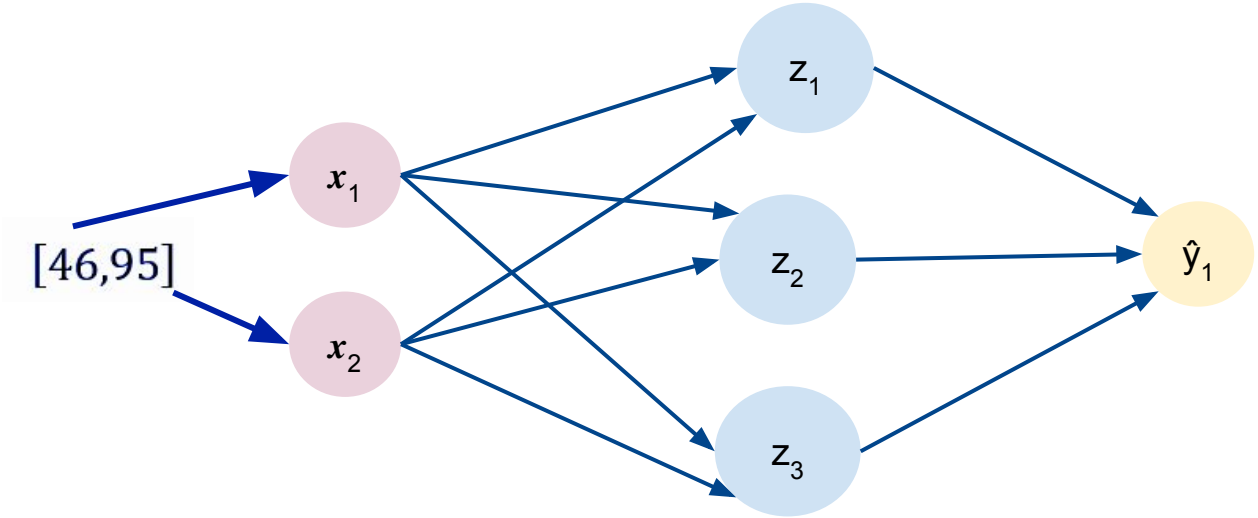
Example



Example

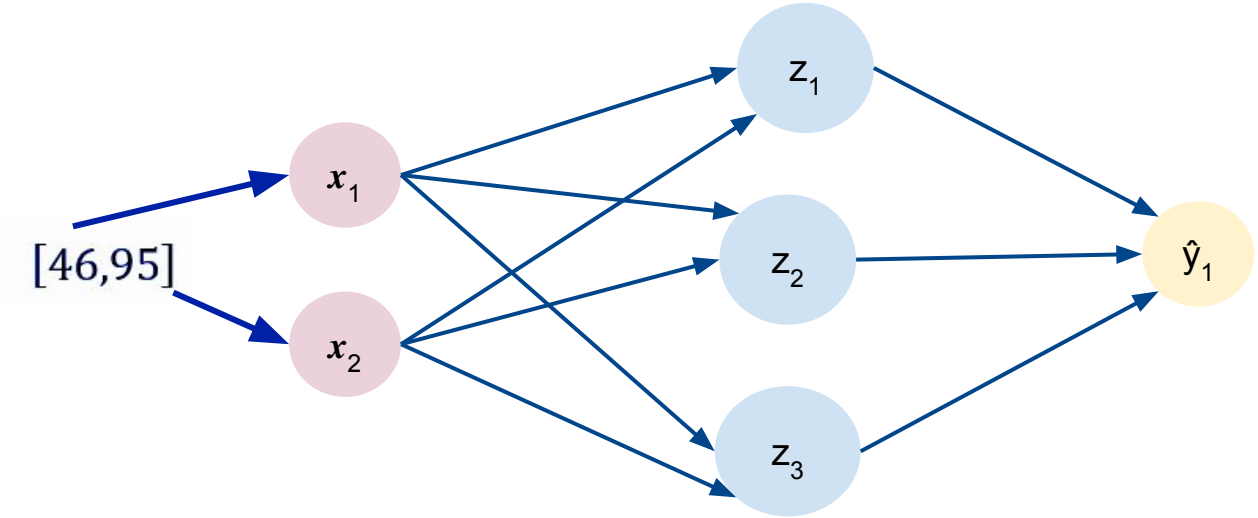

?
$\begin{bmatrix} 46 \\ 95 \end{bmatrix}$
x_2 = Humidity
x_1 = Temperature
Rain
No Rain

Example



Predicted=0.1
(10%)

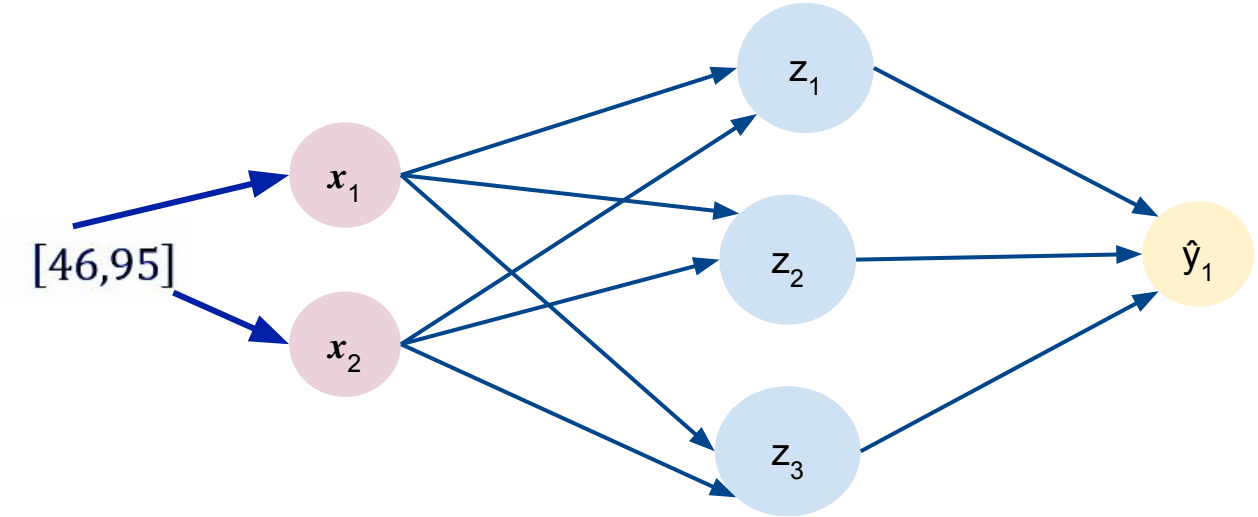
Example



Predicted=0.1 (10%)
Actual=1 (over 50%)

How we quantify the loss?

The loss of out network measures the cost incurred from incorrect predictions



Predicted=0.1 (10%)
Actual=1 (over 50%)

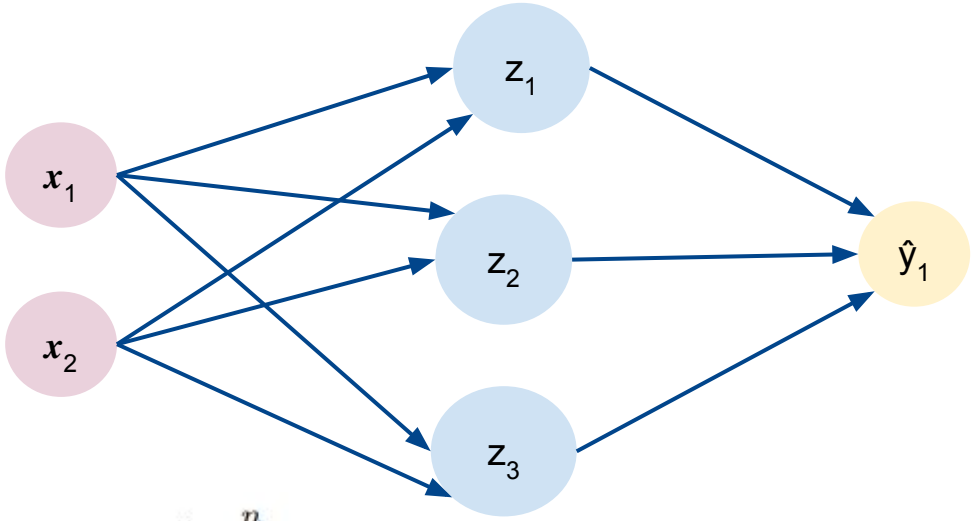
$$\mathcal{L}\left(\underbrace{f\left(\mathbf{x}^{(i)};\mathbf{W}\right)}_{\text{Predicted}},\underbrace{y^{(i)}}_{\text{Actual}}\right)$$

(Amini, 2023)

Empirical Loss

The empirical loss measures the total loss over our entire dataset

$$\begin{bmatrix} 46, & 95 \\ 90, & 70 \\ 85, & 55 \\ & \dots \end{bmatrix}$$



$f(x)$ y

$$\begin{bmatrix} 0.1 \\ 0.8 \\ 0.6 \\ \dots \end{bmatrix} \text{x} \checkmark \text{x} \quad \begin{bmatrix} 1 \\ 0 \\ 1 \\ \dots \end{bmatrix}$$

All known as:
Objective function
Empirical risk
Cost function

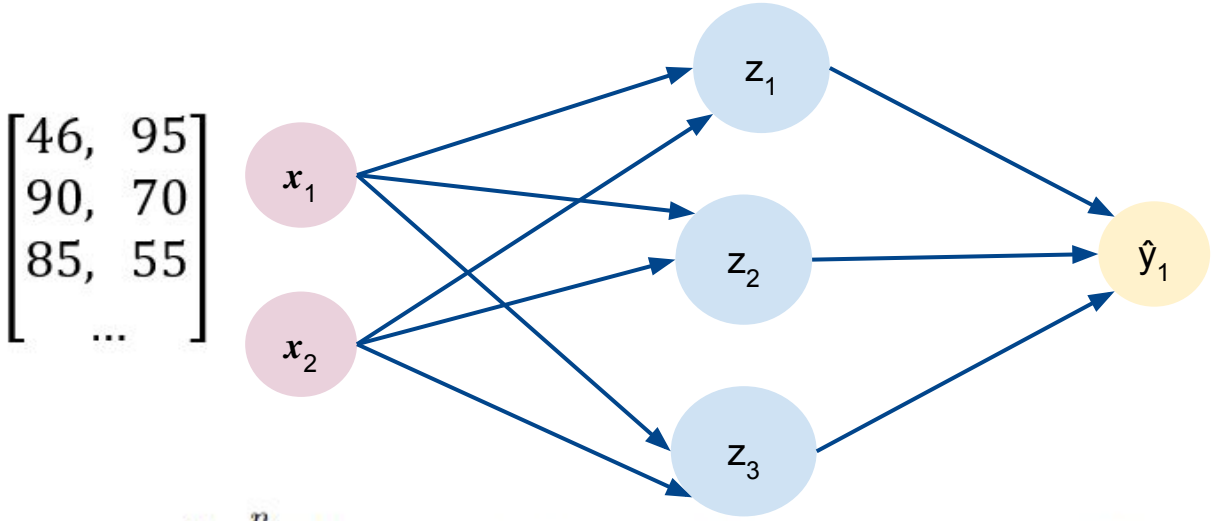
$$J(W) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(f\left(x^{(i)}; ,V\right), y^{(i)}\right)$$

Predicted Actual

(Amini, 2023)

Binary Cross Entropy Loss

Cross entropy loss can be used with models that output a probability between 0 and 1



$f(x)$ y

$\begin{bmatrix} 0.1 \\ 0.8 \\ 0.6 \\ \dots \end{bmatrix}$ x ✔ x $\begin{bmatrix} 1 \\ 0 \\ 1 \\ \dots \end{bmatrix}$

$$J(W) = -\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log\left(f\left(x^{(i)};W\right)\right) + (1-y^{(i)})\log\left(1-f\left(x^{(i)};W\right)\right)\right]$$

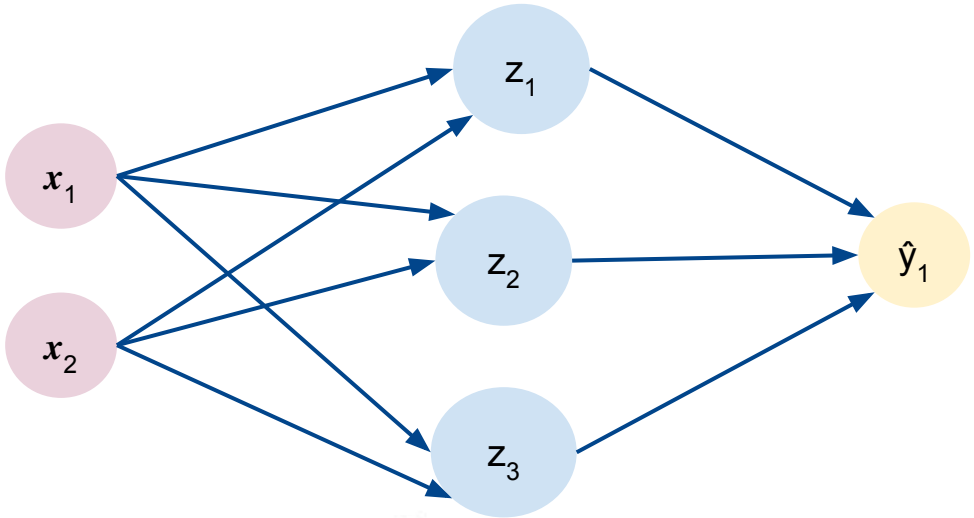
Actual Predicted Actual Predicted

(Amini, 2023)

Mean Squared Error Loss

Mean squared error loss can be used with regression models that output continuous real numbers

$$\begin{bmatrix} 46, & 95 \\ 90, & 70 \\ 85, & 55 \\ & \dots \end{bmatrix}$$



$$\begin{matrix} f(x) & & y \end{matrix}$$

$$\begin{bmatrix} 0.1 \\ 0.8 \\ 0.6 \\ \dots \end{bmatrix} \text{ x } \checkmark \text{ x } \begin{bmatrix} 0.25 \\ 0.33 \\ 0.10 \\ \dots \end{bmatrix}$$

$$J(W) = \frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - f\left(x^{(i)}; W \right) \right)^2$$

Actual Predicted

(Amini, 2023)